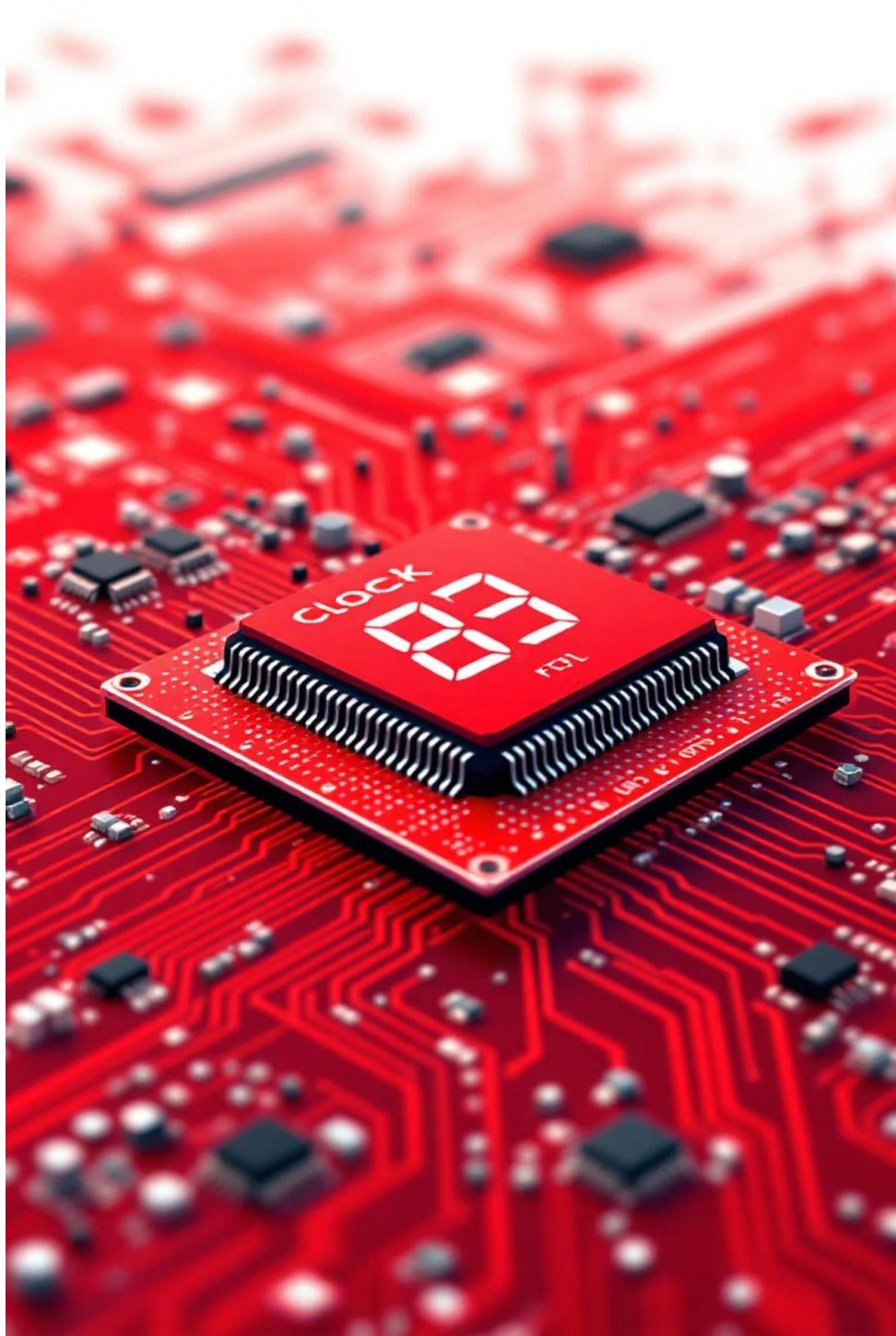
Lógica Secuencial Sincrónica

Fundamentos, análisis y diseño de circuitos digitales con memoria



Ing.Sergio Olmedo

Circuitos Secuenciales: Definición y Estructura

Los circuitos secuenciales representan un avance fundamental respecto a los circuitos combinacionales. Su característica distintiva es que **las salidas no dependen exclusivamente de las entradas actuales**, sino también del historial de estados previos del sistema.

Esta capacidad de "recordar" información convierte a los circuitos secuenciales en elementos indispensables para cualquier sistema digital práctico, desde microprocesadores hasta sistemas de control industrial.



Circuito Combinacional

Procesa las señales de entrada y del estado almacenado



Elementos de Almacenamiento

Memoria que define el estado actual del sistema



Trayectoria de Retroalimentación

Conexión que cierra el ciclo entre memoria y lógica

El **estado** del circuito secuencial en un momento dado queda completamente determinado por la información almacenada en sus elementos de memoria. Las salidas y el siguiente estado son funciones tanto de las entradas externas como del estado actual, creando una sucesión temporal de entradas, salidas y estados internos.

Made with GAMMA

Sincronización mediante Señal de Reloj

Los sistemas secuenciales sincrónicos se caracterizan porque su comportamiento está perfectamente definido en **instantes discretos de tiempo**, determinados por una señal de reloj. Esta sincronización temporal es la clave de su estabilidad y confiabilidad.

01

Generador de Reloj

Produce un tren periódico de pulsos de reloj con frecuencia y ciclo de trabajo precisos

02

Distribución de Pulsos

Los pulsos se distribuyen sincronizadamente por todo el sistema digital

03

Actualización de Flip-Flops

Los elementos de almacenamiento se actualizan únicamente al llegar cada pulso

04

Transición de Estado

El cambio de un estado al siguiente ocurre a intervalos preestablecidos

Ventaja fundamental: Los circuitos secuenciales sincrónicos con reloj casi nunca manifiestan problemas de estabilidad y facilitan enormemente el análisis al dividir su temporización en pasos discretos y predecibles.

Made with GAMMA

Elementos de Almacenamiento: Latches y Flip-Flops

Latches: Operación por Nivel

Los latches son los dispositivos de almacenamiento más básicos, operando con **niveles de señal** en lugar de transiciones. Aunque útiles en ciertos contextos, no son prácticos para circuitos secuenciales sincrónicos.

- **Latch SR:** Implementable con compuertas NOR o NAND. Estados: establecido (Q=1) y restablecido (Q=0). Presenta estado indefinido cuando ambas entradas son activas simultáneamente
- **Latch D (Transparente):** Elimina el estado indefinido garantizando que S y R nunca sean 1 simultáneamente. Cuando C=1, la salida Q sigue los cambios en D

Flip-Flops: Disparo por Borde

Los flip-flops son dispositivos diseñados específicamente para circuitos con reloj. Su característica fundamental es el **disparo por borde**: se activan únicamente durante una transición de la señal de reloj.

- **Flip-Flop D:** El más económico y eficiente. Ecuación característica: $Q(t+1)=D$
- **Flip-Flop JK:** Versátil: establece, restablece y complementa. Ecuación: $Q(t+1)=JQ' + K'Q$
- **Flip-Flop T:** Complementador, ideal para contadores. Ecuación: $Q(t+1)=T \ loplus \ Q$

Los flip-flops requieren cumplir estrictos **parámetros de tiempo**: el tiempo de preparación (*setup*) y el tiempo de retención (*hold*) especifican cuándo debe ser estable la entrada D respecto a la transición de reloj. Además, pueden incluir **entradas directas asincrónicas** (preset, clear) que fuerzan un estado conocido independientemente del reloj.

Made with GAMMA

Análisis: Formalismos Algebraicos

El análisis de circuitos secuenciales busca comprender y predecir su comportamiento temporal. Las representaciones algebraicas constituyen la base matemática fundamental para este análisis.

1

Ecuaciones de Estado (Transición)

Especifican el **siguiente estado** en función del estado actual y las entradas. El miembro derecho es una expresión booleana que indica las condiciones para que el siguiente estado sea 1.

Ejemplo: $A(t+1)=Ax+Bx$

Para flip-flops D, estas ecuaciones se deducen directamente de las compuertas que alimentan la entrada D.

2

Ecuaciones de Entrada (Excitación)

Describen la **parte combinacional** que genera las entradas a los flip-flops. El conocimiento de estas ecuaciones y del tipo de flip-flop es esencial para construir el diagrama lógico.

Para el flip-flop D, la ecuación de entrada (D_Q) es idéntica a la ecuación de estado correspondiente ($Q(t+1)$).

3

Ecuaciones de Salida

Expresiones booleanas que describen las **salidas externas** del circuito en función del estado actual y, posiblemente, de las entradas.

Ejemplo: $y=(A+B)x'$

Estas ecuaciones determinan cómo el circuito se comunica con el exterior.

Made with GAMMA

Análisis: Representaciones Tabular y Gráfica

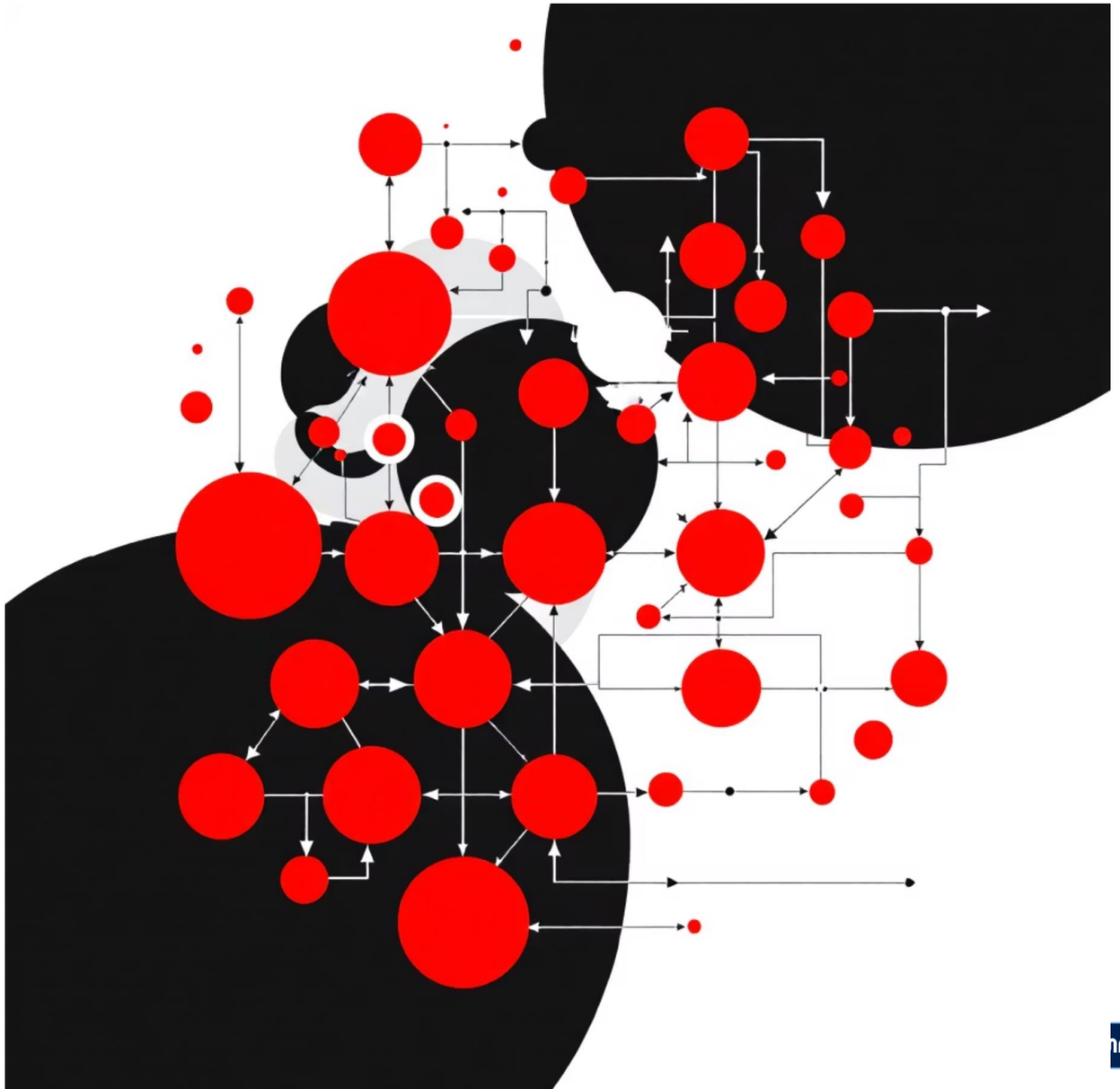
Tabla de Estados

Presenta de forma compacta y sistemática la sucesión temporal del circuito. Se organiza en cuatro secciones fundamentales:

- **Estado actual**: Configuración presente de los flip-flops
- **Entrada**: Valores de las señales externas
- **Siguiente estado**: Configuración tras el pulso de reloj
- **Salida**: Valores producidos por el circuito

Un circuito con *m* flip-flops y *n* entradas requiere *2^{m+n}* filas para representar todas las combinaciones posibles.

Diagrama de Estados



Modelos de Máquinas de Estado: Mealy vs Moore

Las Máquinas de Estados Finitos (FSM) se clasifican en dos modelos fundamentales según cómo generan sus salidas. Esta distinción tiene implicaciones importantes en el diseño y el comportamiento temporal del circuito.

Modelo Mealy

Dependencia de salida: Las salidas son función tanto del **estado actual como de las entradas**.

Comportamiento temporal: Las salidas pueden cambiar si las entradas varían durante el ciclo de reloj, pudiendo presentar valores transitorios falsos.

Notación en diagrama: Los valores de entrada y salida se marcan sobre las flechas de transición, separados por una diagonal (entrada/salida).

Ventaja: Generalmente requiere menos estados para implementar la misma funcionalidad.

Modelo Moore

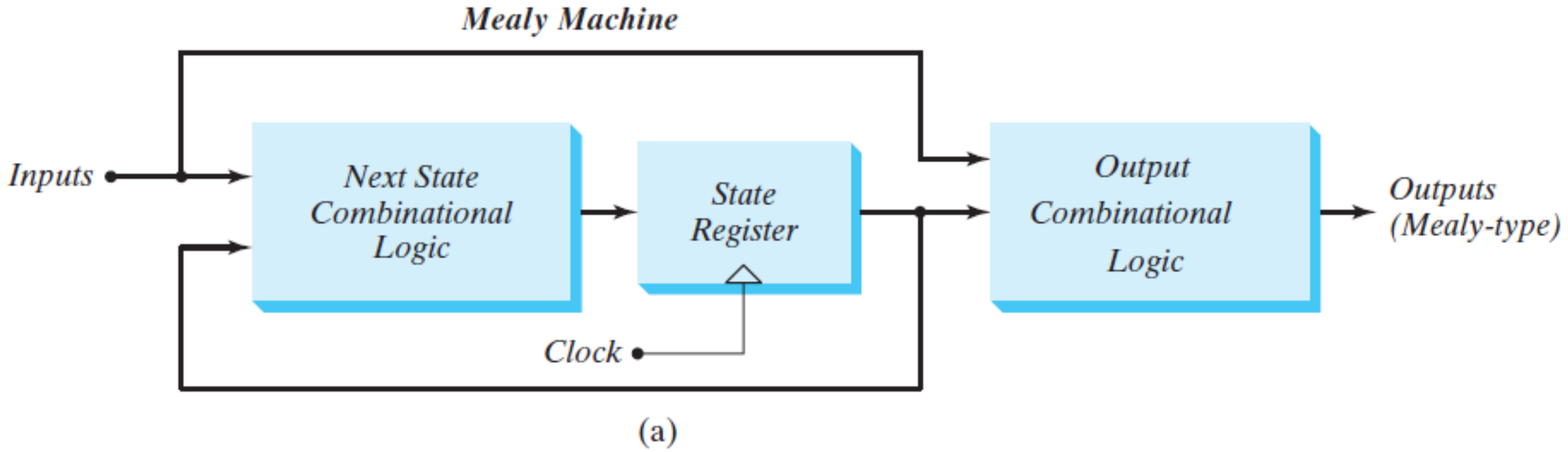
Dependencia de salida: Las salidas son función **solamente del estado actual**, independientes de las entradas.

Comportamiento temporal: Las salidas están perfectamente sincronizadas con el reloj, ya que solo dependen de las salidas de los flip-flops.

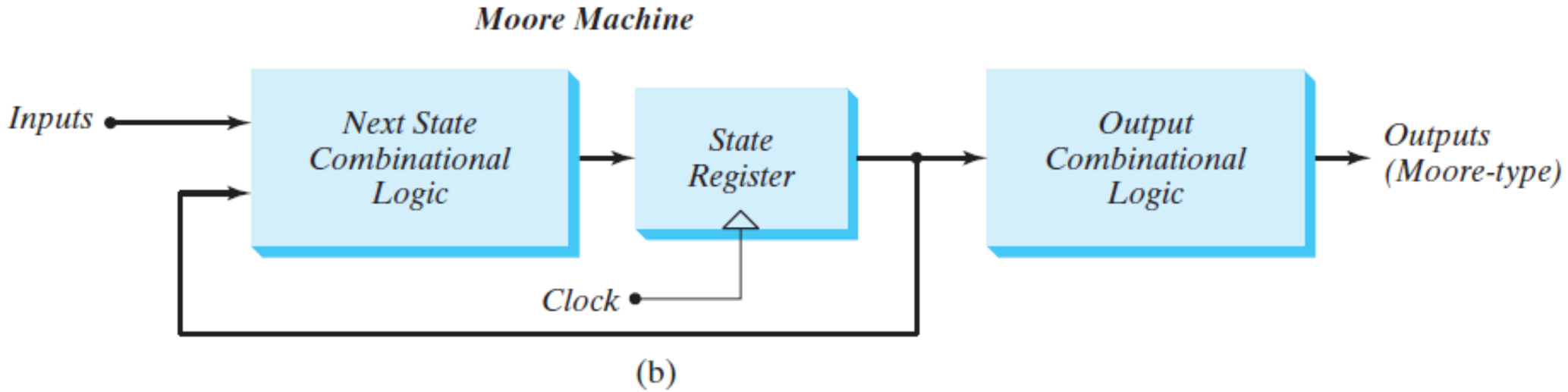
Notación en diagrama: La salida se indica **dentro del círculo** de estado, junto con la identificación del estado.

Ventaja: Mayor estabilidad y predictibilidad de las salidas, sin glitches durante transiciones.

MÁQUINA DE ESTADOS: MEALY Y MOORE

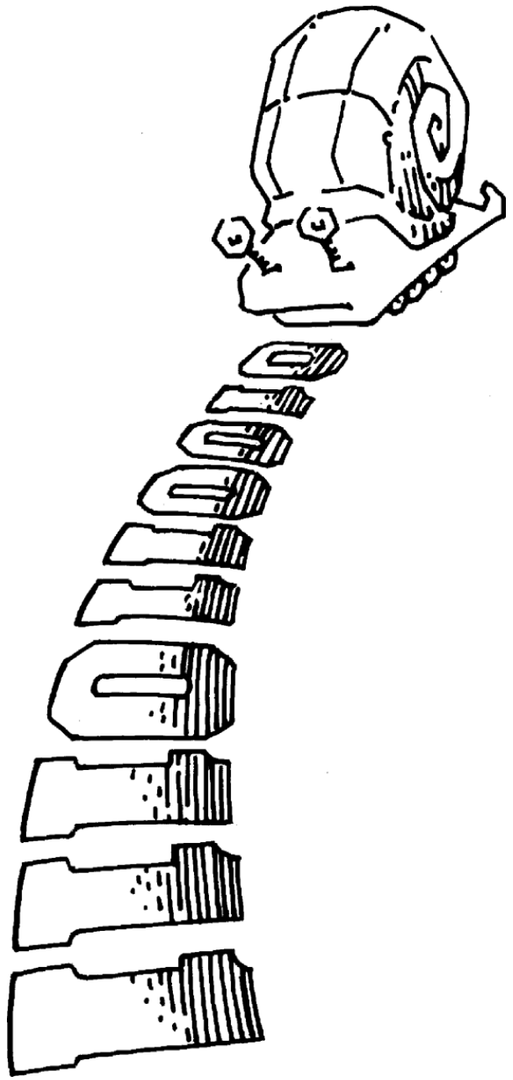


(a)



(b)

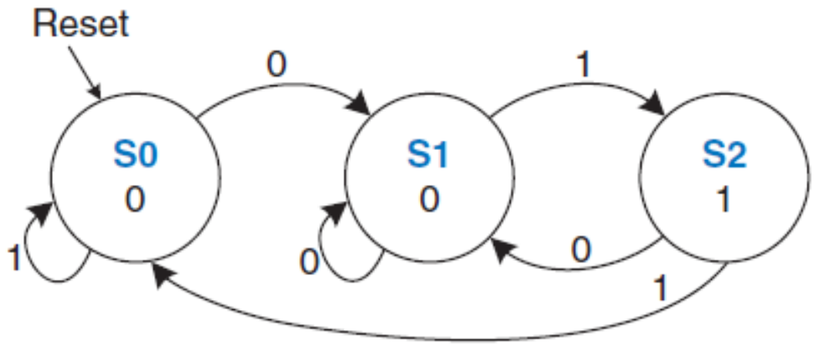
EJEMPLO COMPARATIVO MOORE-MEALY



Alyssa P. Hacker posee un caracol robotizado de mascota con un cerebro diseñado con una FSM. El caracol se arrastra de izquierda a derecha a lo largo de una cinta de papel que contiene una secuencia de 1s y 0s. En cada ciclo de reloj, el caracol se arrastra hasta el siguiente bit. El caracol sonríe cuando los dos últimos bits sobre los que se ha arrastrado son, de izquierda a derecha, 01. Diseñar la FSM para calcular cuando el caracol debe sonreír. La entrada A es el bit debajo de las antenas del caracol. La salida Y es VERDADERA cuando el caracol sonríe. Comparar los diseños de máquinas de estado de Moore y Mealy. Dibuje un diagrama de tiempo para cada máquina de estado que muestre la entrada, los estados y la salida cuando el caracol de Alyssa se arrastra a lo largo de la secuencia 0100110111.

Fuente: Dr Toledo

MÁQUINA DE MOORE



(a)

Table 3.11 Moore state transition table

Current State S	Input A	Next State S'
S0	0	S1
S0	1	S0
S1	0	S1
S1	1	S2
S2	0	S1
S2	1	S0

Table 3.12 Moore output table

Current State S	Output Y
S0	0
S1	0
S2	1

MÁQUINA DE MOORE

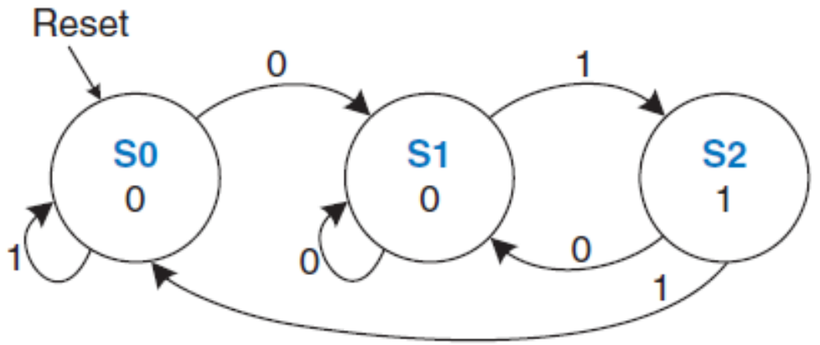


Table 3.13 Moore state transition table with state encodings

Current State S_1	Current State S_0	Input A	Next State S_1'	Next State S_0'
0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0

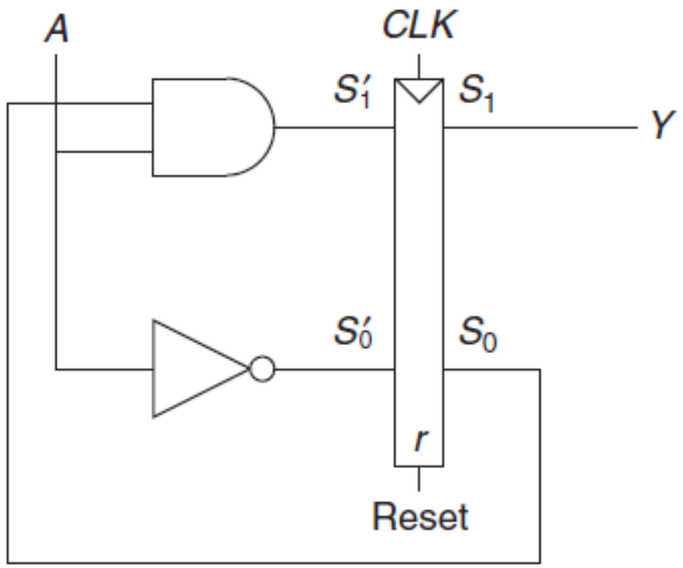
$$S_1' = S_0 A$$
$$S_0' = \overline{A}$$

$$Y = S_1$$

Table 3.14 Moore output table with state encodings

Current State S_1	Current State S_0	Output Y
0	0	0
0	1	0
1	0	1

MÁQUINA DE MOORE: IMPLEMENTACIÓN



$$S_1' = S_0 A$$
$$S_0' = \overline{A}$$

$$Y = S_1$$

MÁQUINA DE MEALY

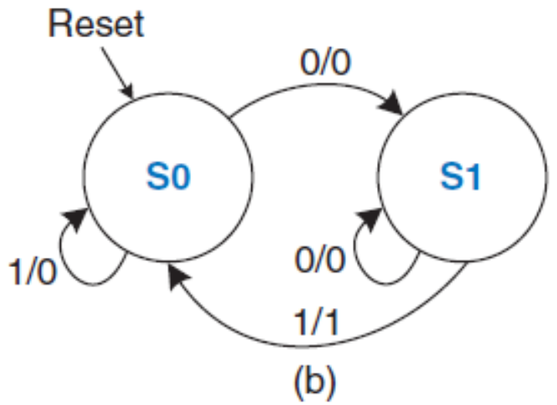


Table 3.15 Mealy state transition and output table

Current State S	Input A	Next State S'	Output Y
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

MÁQUINA DE MEALY

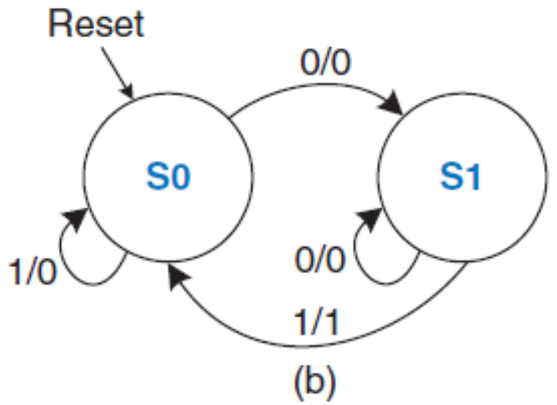


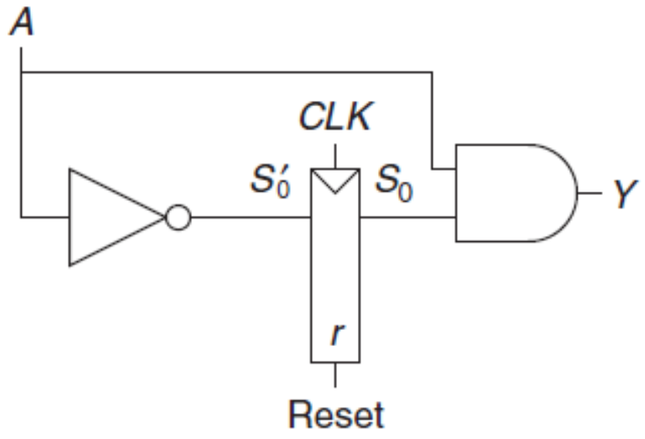
Table 3.16 Mealy state transition and output table with state encodings

Current State S_0	Input A	Next State S_0'	Output Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

$$S_0' = \overline{A}$$

$$Y = S_0 A$$

MÁQUINA DE MEALY: IMPLEMENTACIÓN

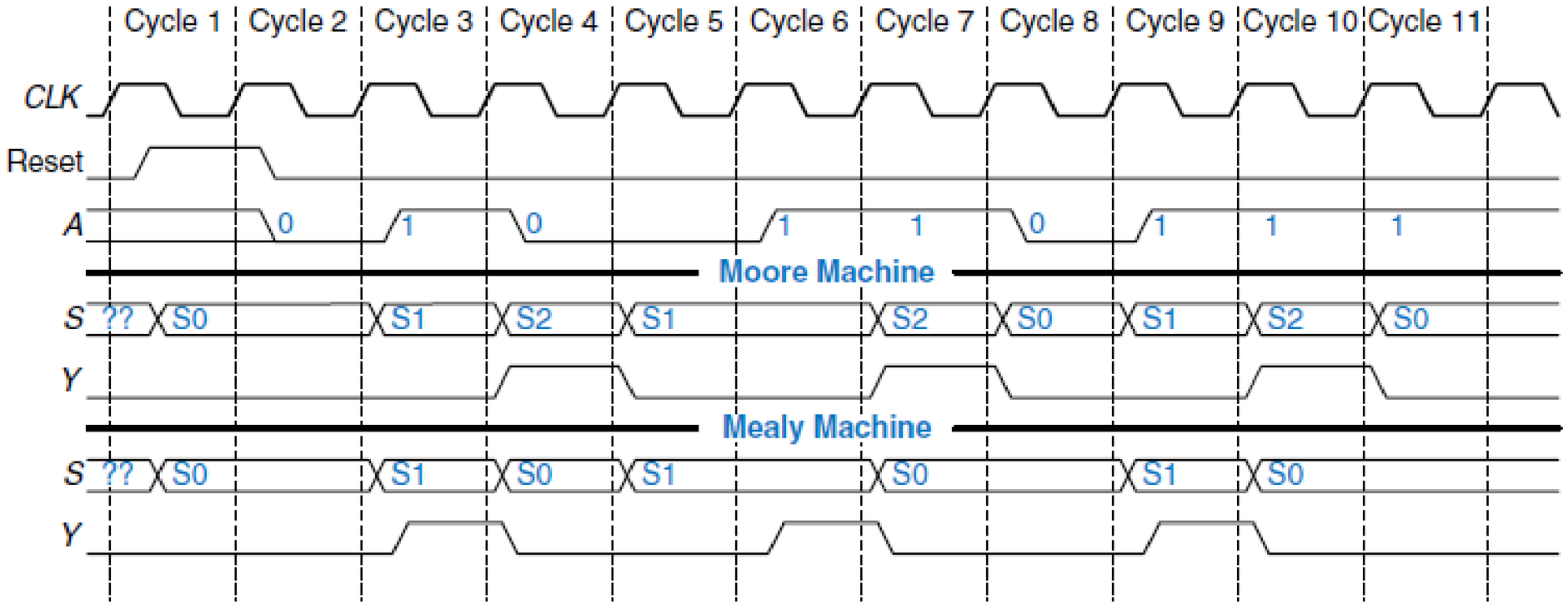


(b)

$$S_0' = \overline{A}$$

$$Y = S_0 A$$

DIAGRAMAS MÁQUINAS DE MEALY Y MOORE



HDL PARA MÁQUINAS DE MEALY Y MOORE

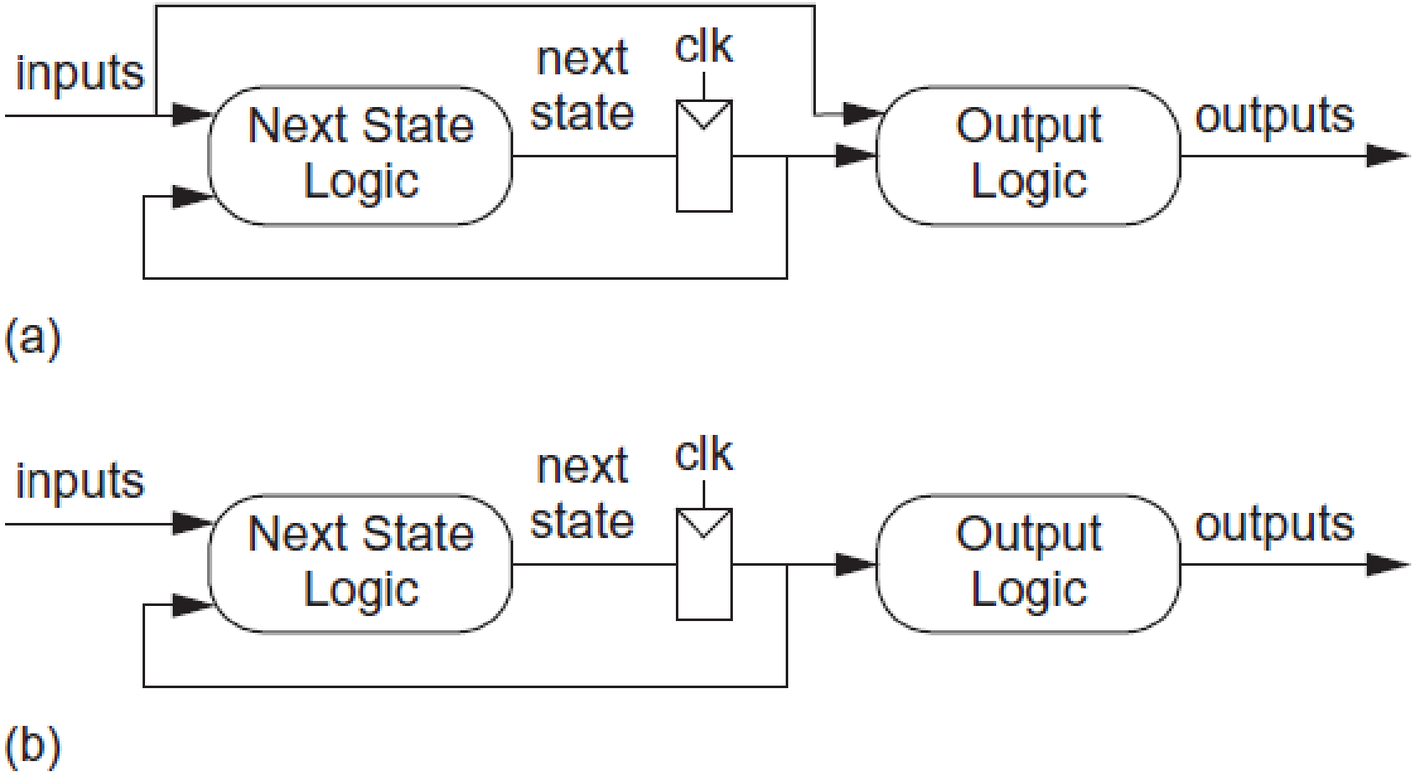
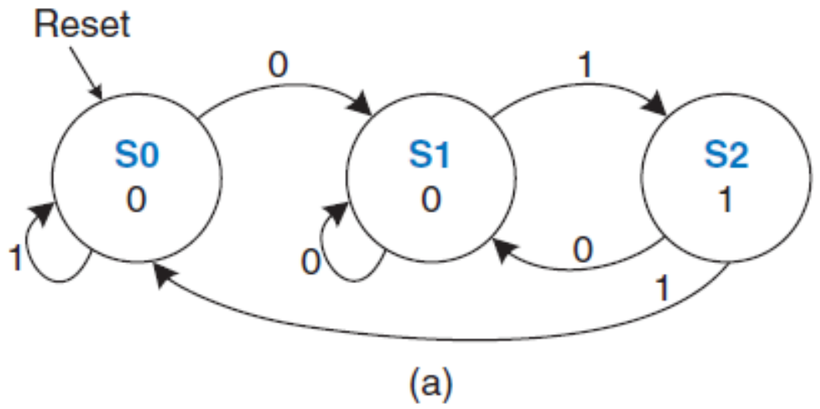


FIGURE A.32 Mealy and Moore machines

HDL PARA MÁQUINA DE MOORE



(a)

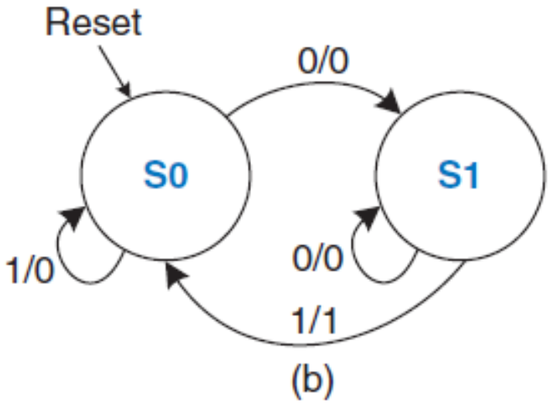
```
module patternMoore ( input clk,
                      input reset,
                      input a,
                      output y);

  reg [1:0] state,  nextstate;

  parameter S0 = 2'b00;
  parameter S1 = 2'b01;
  parameter S2 = 2'b10;
```

```
// state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;
  // next state logic
  always @ (*)
    case (state)
      S0: if (a) nextstate = S0;
          else   nextstate = S1;
      S1: if (a) nextstate = S2;
          else   nextstate = S1;
      S2: if (a) nextstate = S0;
          else   nextstate = S1;
      default:  nextstate = S0;
    endcase
 // output logic
  assign y = (state==S2);
endmodule
```

HDL PARA MÁQUINA DE MEALY




```
module patternMealy ( input clk,
                      input reset,
                      input a,
                      output y);
  reg  state, nextstate;
  parameter S0 = 1'b0;
  parameter S1 = 1'b1;
```

```
  // state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next state logic
  always @(*)
    case (state)
      S0: if (a) nextstate = S0;
          else   nextstate = S1;
      S1: if (a) nextstate = S0;
          else  nextstate = S1;
      default:  nextstate = S0;
    endcase

  // output logic
    assign y = (a & state==S1);
endmodule
```

HDL PARA MÁQUINA DE MOORE

```
module divideby3FSM(input  clk,
                    input  reset,
                    output y);

  reg [1:0] state, nextstate;

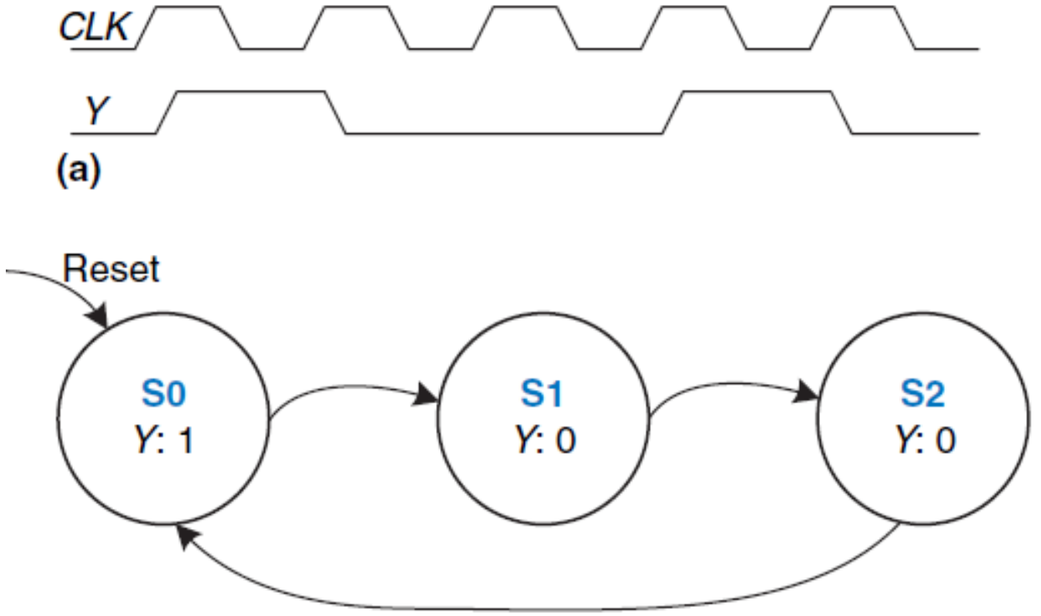
  parameter S0 = 2'b00;
  parameter S1 = 2'b01;
  parameter S2 = 2'b10;

  // state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next state logic
  always @ (*)
    case (state)
      S0: nextstate = S1;
      S1: nextstate = S2;
      S2: nextstate = S0;
      default: nextstate = S0;
    endcase

  // output logic
  assign y = (state == S0);
endmodule
```

FIGURE A.33 Divide-by-3 counter state transition diagram



Procedimiento de Diseño de Circuitos Secuenciales

El diseño sistemático de circuitos secuenciales sincrónicos transforma las especificaciones funcionales en un circuito físico implementable. Este proceso metódico garantiza resultados correctos y optimizados.

Deducir el Diagrama de Estados

Paso más creativo y difícil. Requiere comprender profundamente el comportamiento deseado y traducirlo en estados y transiciones.

Reducir el Número de Estados

Eliminar estados equivalentes que producen las mismas salidas y transiciones. Reduce complejidad sin alterar funcionalidad.

Asignar Valores Binarios

Codificar cada estado con una combinación binaria única. La elección afecta la complejidad del circuito resultante.

Obtener Tabla de Estados Codificada

Reescribir la tabla de estados usando las asignaciones binarias. Base para las ecuaciones lógicas.

Escoger Tipo de Flip-Flops

Seleccionar D, JK o T según disponibilidad, costo y facilidad de implementación.

Simplificar Ecuaciones

Usar mapas de Karnaugh o métodos algebraicos para minimizar las ecuaciones de entrada y salida.

Dibujar Diagrama Lógico

Representación final con compuertas, flip-flops y conexiones listas para implementación física.

Estados no utilizados: Se tratan como condiciones de indiferencia durante la simplificación, permitiendo mayor optimización del circuito combinacional.

Made with GAMMA

Tablas de Excitación y Síntesis con Flip-Flops

Las tablas de excitación son herramientas fundamentales cuando se diseñan circuitos con flip-flops JK o T, permitiendo determinar qué entradas se requieren para lograr cada transición de estado deseada.

Concepto y Aplicación

Una tabla de excitación muestra las **condiciones de entrada requeridas** para que un flip-flop transite de un estado actual *Q(t)* a un siguiente estado *Q(t+1)*. Esto invierte la lógica del análisis: en lugar de determinar el próximo estado dadas las entradas, se determinan las entradas necesarias para alcanzar el próximo estado deseado.

Ventajas del Flip-Flop JK

El flip-flop JK es especialmente ventajoso en diseño porque su tabla de excitación incluye numerosas condiciones de indiferencia (X). Esto ocurre porque muchas transiciones pueden lograrse independientemente del valor de J o K.



Por ejemplo, para transitar de Q=0 a Q=1, se requiere J=1, pero K puede ser 0 o 1 (indiferente). Esta flexibilidad genera múltiples términos X en los mapas de Karnaugh.

Simplificación Mejorada

Las abundantes condiciones de indiferencia permiten **mayor simplificación** con mapas de Karnaugh, resultando en circuitos combinacionales más sencillos y económicos.

Flip-Flop T para Contadores

El flip-flop T requiere T=1 para complementar el estado y T=0 para mantenerlo. Los **contadores binarios** se construyen eficientemente con flip-flops T por su naturaleza de alternancia.

Made with GAMMA

Descripción y Síntesis mediante HDL (Verilog)

Los Lenguajes de Descripción de Hardware (HDL) como Verilog permiten modelar, simular y sintetizar circuitos secuenciales de forma eficiente y profesional.

1 Modelado de Comportamiento con Always

Los bloques `always` describen el comportamiento del circuito. El control por suceso usa `posedge` (borde positivo) o `negedge` (borde negativo) para detectar transiciones de reloj.

Las asignaciones pueden ser bloqueadoras (`=`) o no bloqueadoras (`<=`). Las no bloqueadoras son indispensables para circuitos secuenciales, evitando condiciones de carrera.

2 Descripción de Flip-Flops

Un flip-flop D se describe con: `always @(posedge CLK) Q <= D;`

Para flip-flops con reset asincrónico: `always @(posedge CLK or negedge RST)`, incluyendo el evento asincrónico en la lista de sensibilidad.

3 Modelado de Máquinas de Estado

Se pueden usar tres bloques always concurrentes: uno para la transición sincrónica de estado (en el borde de reloj), otro para calcular el siguiente estado (lógica combinacional), y un tercero para evaluar las salidas.

4 Descripción Estructural

Combina bloques `always` para flip-flops con enunciados `assign` para ecuaciones booleanas de la lógica combinacional. Los módulos se interconectan mediante creación de ejemplares (instanciación).

Esta metodología de diseño con HDL facilita la verificación, simulación y síntesis automatizada de circuitos secuenciales complejos, constituyendo el estándar industrial moderno para el diseño de sistemas digitales.

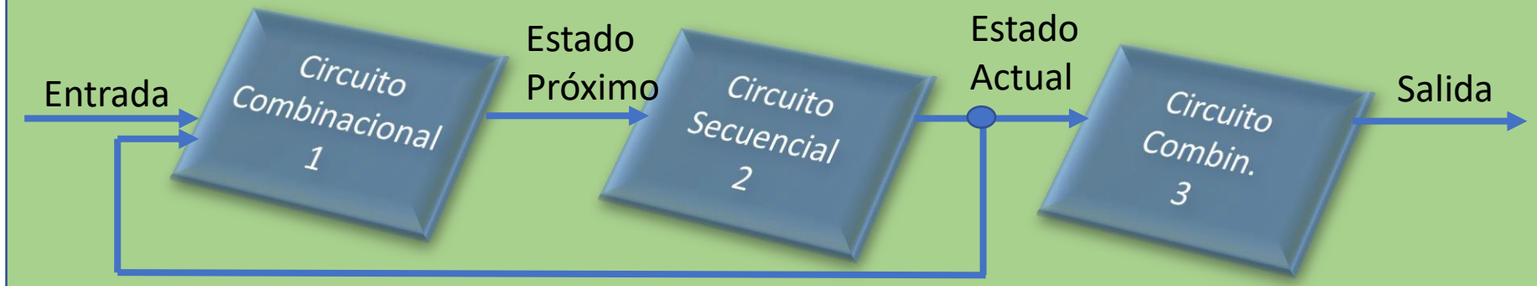
Made with GAMMA

Descripción HDL (Verilog) de una Máquina de Moore (Binary)

```
parameter <state1> = 2'b00;
parameter <state2> = 2'b01;
parameter <state3> = 2'b10;
parameter <state4> = 2'b11;

(* FSM_ENCODING="SEQUENTIAL", SAFE_IMPLEMENTATION="NO" *) reg
[1:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
    state <= <state1>;
    <outputs> <= <initial_values>;
  end
```



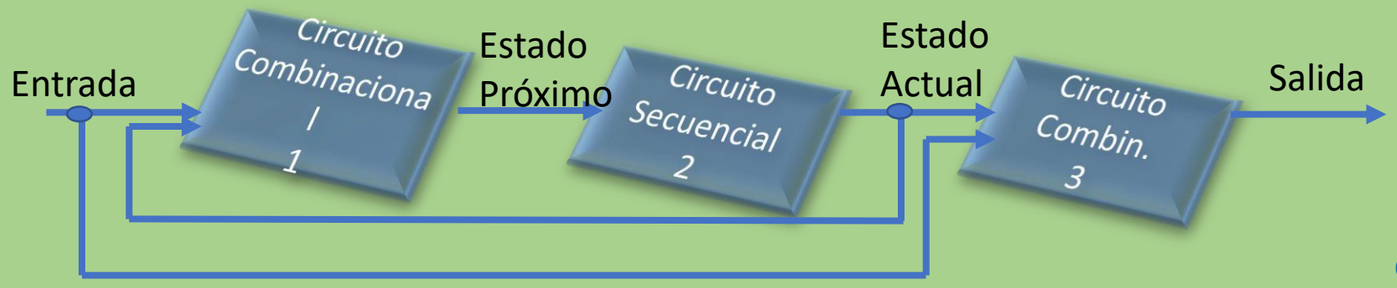

```
else
    (* FULL_CASE, PARALLEL_CASE *) case (state)
      <state1> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
        <outputs> <= <values>;
      end
      <state2> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
        <outputs> <= <values>;
      end
      <state3> : begin
          ....
      <state4> : begin
          ....
endcase
```

Descripción HDL (Verilog) de una Máquina de Mealy (binary)

```
parameter <state1> = 2'b00;
parameter <state2> = 2'b01;
parameter <state3> = 2'b10;
parameter <state4> = 2'b11;

(* FSM_ENCODING="SEQUENTIAL", SAFE_IMPLEMENTATION="NO" *) reg
[1:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
    state <= <state1>;
    <outputs> <= <initial_values>;
  end
```




```
else
(* FULL_CASE, PARALLEL_CASE *) case (state)
        <state1> : begin
          if (<condition>)
            state <= <next_state>;
          else if (<condition>)
            state <= <next_state>;
          else
            state <= <next_state>;
        end
        <state2> : begin
          if (<condition>)
            state <= <next_state>;
          else if (<condition>)
            state <= <next_state>;
          else
            state <= <next_state>;
        end
        <state3> : begin
            .....
        <state4> : begin
         ......
  endcase

  assign <output1> = <logic_equation_based_on_states_and_inputs>;
  assign <output2> = <logic_equation_based_on_states_and_inputs>;
  // Add other output equations as necessary
```

Preguntas



Made with GAMMA